

Trabalho final

Relatório de Grupo

Técnicas Avançadas de Programação 3D

Realizado Por:

Gonçalo Araújo - 27928

Gonçalo Veloso - 22348

João Ribeiro - 27926

Tiago Miranda - 27937

Conteúdo

1. Introdução e Tema.....	6
2. Visão Geral dos Shaders.....	6
3. Hologram.....	7
3.1 Descrição e Intenção	7
3.2 Pipeline.....	7
3.3 Vertex Shader — Breathing e Glitch	8
3.4 Fragment Shader — Rim, Scanlines e Flicker	8
3.5 Integração C# — HologramController.....	9
3.6 Built-in Variables e Funções CG.....	9
4. Energy Shield.....	11
4.1 Descrição e Intenção	11
4.2 Pipeline.....	11
4.3 Vertex Shader.....	11
4.4 Geometry Shader — Ripple Geométrico	12
4.5 Fragment Shader — Fresnel e Ripple de Cor	13
4.6 Integração C# — EnergyShieldController.....	14
4.7 Built-in Variables e Funções CG.....	14
5. CCTV Glitch (Post-Process).....	15
5.1 Descrição e Intenção	15
5.2 Pipeline.....	16
5.3 Função random — Conversão de GLSL	16
5.4 Abertura Cromática.....	17
5.5 Scanlines, Grão e Desaturação.....	17
5.6 Integração C# — CCTVGlitchEffect.....	17
5.7 Built-in Variables e Funções CG.....	18

6. Reading Steiner (Post-Process)	19
6.1 Descrição e Intenção	19
6.2 Screen Tearing.....	19
6.3 RGB Split e Ruído	19
6.4 Inversão de Cor	20
6.5 Integração C# — ReadingSteinerEffect e MicrowaveReadingSteinerDriver	20
6.6 Built-in Variables e Funções CG.....	21
7. Kerr — Buraco Negro Volumétrico	21
7.1 Descrição e Intenção	21
7.2 Otimização do Pipeline e Isolamento de Espaço de Cálculo.....	22
7.3 Motor de Raymarching e Integração Vetorial Gravitacional	23
7.4 Disco de Acréscimo.....	24
7.5 Output Final e Distorção do Fundo	25
7.6 Integração C# — BlackHoleActivationButton e SpinUp	25
7.7 Built-in Variables e Funções CG.....	26
8. Radioactive Liquid	27
8.1 Descrição e Intenção	27
8.2 Arquitetura Multi-Pass com Stencil.....	27
8.3 Vertex Shader (Pass Liquid).....	28
8.4 Smooth Noise — Conversão de GLSL.....	28
8.5 Fragment Shader — Ondas, Clip, Espuma e Glow	29
8.6 Built-in Variables e Funções CG.....	30
9. Stencil Portal e Stencil World (X-Ray)	31
9.1 Descrição e Intenção	31
9.2 StencilPortal — Shader "Custom/Stencil/PortalMask_Hologram"	32
9.3 StencilWorld — Shader "Custom/Stencil/HiddenObject_XRay"	32

9.4 Built-in Variables e Funções CG.....	33
10. Técnicas Consideradas e Não Implementadas	34
10.1 Interior Mapping	34
10.2 Triplanar Mapping.....	34
10.3 Depth Texture	34
10.4 Hull Shader (Tesselação)	34
11. Dificuldades e Soluções.....	35
11.1 Geometry Shader e #pragma target 4.0	35
11.2 Ordem dos Passes no RadioactiveLiquid.....	35
11.3 GrabPass e Compatibilidade	35
11.4 Conversões GLSL → HLSL	35
11.5 Integração XR e Cadeia de Scripts.....	35
11.6 CharacterController e Teleporte	35
12. Portal Dimensional — The Casting of Frank Stone (Magic Circle)	36
12.1 Identificação do Efeito no Jogo	36
12.1.1 Camadas Visuais Identificadas.....	36
12.2 Teoria — Técnicas de Implementação	37
12.2.1 Borda — Fresnel	37
12.2.2. Interior — FBM e Rotação de UV.....	37
12.2.3 Fragmentos em Órbita — Value Noise	38
12.2.4 Núcleo Central	38
12.2.5 Alpha Border.....	39
12.3 Implementação	39
13. Testes em Hardware Real – Meta Quest (Android).....	40
13.1 Kerr – GrabPass não suportado em OpenGL ES.....	40
13.2 EnergyShield – Geometry Shader ignorado	41

13.3 CCTV Glitch e ReadingSteiner – Post-process ausente	42
13.4 StencilPortal – Ordem de draw calls instável.....	43
13.5 Cena Geral	44
14. Conclusão	45

1. Introdução e Tema

O presente trabalho insere-se na unidade curricular de Introdução à Programação 3D e consiste na criação de um pack de shaders coeso para o motor Unity, escrito em HLSL/Cg sobre a Built-in Render Pipeline (BiRP). O pack é acompanhado de uma demonstração interativa em VR.

O tema escolhido é um laboratório inspirado em Steins; Gate. A narrativa explora viagens no tempo, linhas de universo alternativas e anomalias científicas. Esta pressuposto justifica uma escolha vasta e coerente de efeitos: os hologramas, o escudo de plasma do laboratório, o buraco negro volumétrico que se forma dentro do microondas, os frascos de líquido radioativo, o portal que permite ver através das linhas de universo, e os efeitos de pós-processamento que simulam a desintegração da realidade no momento de um salto temporal.

Todos os shaders foram escritos de raiz em HLSL, sem usar surface shaders. Esta decisão foi intencional: o nível de controlo necessário para implementar um raymarcher volumétrico, um geometry shader de impacto ou um sistema de stencil multi-pass não é compatível com a abstração que os surface shaders oferecem.

2. Visão Geral dos Shaders

O pack contém sete shaders originais, cobrindo os vários tipos e técnicas pedidos no enunciado:

- **Hologram** — Vertex + Fragment | Rim/Fresnel, Scanlines animadas, Glitch de vértice por proximidade
- **EnergyShield** — Vertex + Geometry + Fragment | Fresnel, Ripple geométrico por impacto físico
- **CCTV Glitch** — Fragment Post-Process | Hash noise, Abertura cromática, Grão, Desaturação BT.601
- **ReadingSteiner** — Fragment Post-Process | Screen Tear, RGB Split, Inversão de cor
- **Kerr** — Fragment Raymarcher | Gravitational Lensing, Disco de Acrensão, Doppler Relativístico, GrabPass
- **RadioactiveLiquid** — Fragment multi-pass | Stencil, Wave surface, Smooth Noise procedural, Fresnel
- **StencilPortal + StencilWorld** — Fragment | Stencil write/read, ZTest Always, Fresnel, Scanlines X-Ray

3. Hologram

3.1 Descrição e Intenção

O shader Hologram simula a projeção holográfica do Phone Microwave de Rintarou Okabe. O efeito combina três componentes visuais: um contorno luminoso (rim/Fresnel) que torna o objeto mais brilhante nas bordas e quase invisível no centro, linhas de scan horizontais animadas que percorrem o objeto verticalmente, e um glitch de vértice que deforma a geometria de forma intermitente. A intensidade do glitch aumenta conforme o jogador se aproxima do holograma, simulando a interferência eletromagnética causada pela presença humana.

A escolha de implementar o glitch no vertex shader, e não no fragment, é deliberada: deslocar vértices cria deformação geométrica real visível de qualquer ângulo de câmara. Uma distorção de UV no fragment pareceria plana e artificial.

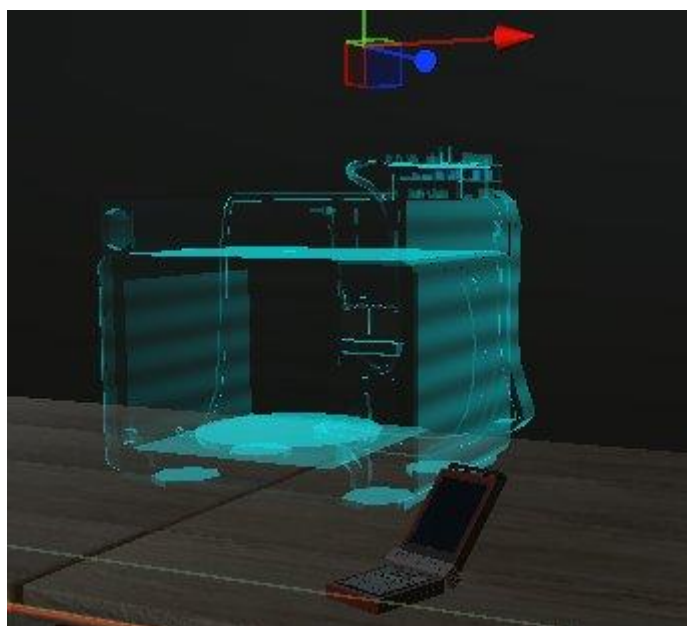


Figura — Hologram shader aplicado ao Phone Microwave na cena

3.2 Pipeline

O shader tem um único passe com vertex e fragment shader. É transparente (Queue=Transparent, Blend SrcAlpha OneMinusSrcAlpha, ZWrite Off) e usa Cull Back para não renderizar o interior do objeto.

3.3 Vertex Shader — Breathing e Glitch

O vertex shader é responsável por dois efeitos de deformação geométrica.

O breathing desloca cada vértice ao longo da sua normal, combinando uma onda seno e um cosseno com velocidades diferentes. Se fosse só sin o movimento seria perfeitamente periódico e artificial. A combinação sin + cos com fator 0.8 cria um movimento orgânico que nunca repete exatamente:

```
float breath = (sin(_Time.y * _BreathSpeed) + cos(_Time.y * _BreathSpeed * 0.8)) * 0.5;
v.vertex.xyz += v.normal * breath * _BreathAmp;
```

O glitch desloca os vértices horizontalmente com base numa onda de alta frequência em Y, que cria faixas horizontais distintas. O step sobre uma senoide lenta decide se o glitch está ativo naquele momento. A variável `_IsConstantGlitch`, enviada pelo script C#, usa `max()` em vez de um `if/else` para ativar o glitch permanente sem branch na GPU:

```
float slice = sin(_Time.y * 50.0 + v.vertex.y * 20.0);

float baseSnap = step(0.8, sin(_Time.y * 15.0));
float glitchSnap = max(baseSnap, _IsConstantGlitch);

v.vertex.x += slice * glitchSnap * _GlitchIntensity * 0.1;
```

3.4 Fragment Shader — Rim, Scanlines e Flicker

O fragment shader calcula o visual final de cada pixel. O rim (Fresnel simplificado) é obtido pelo dot product entre a normal e a direção de vista: quando a normal aponta para a câmara (centro do objeto) o dot é alto e o rim é baixo; nas bordas o dot é próximo de zero e o rim sobe para 1. O pow controla a curvatura — valores altos concentram o brilho numa linha fina nas bordas:

```
float3 viewDir = normalize(UnityWorldSpaceViewDir(i.worldPos));
float3 normal = normalize(i.normal);

float rim = 1.0 - saturate(dot(normal, viewDir));
float rimIntensity = pow(rim, _RimPower);
```

As scanlines são geradas por seno sobre a coordenada Y em world space, animado com `_Time.y`, mapeado de `[-1,1]` para `[0,1]`. O lerp de `_ScanlineIntensity` deixa o artista misturar a intensidade sem alterar o código:


```
float scanline = sin(i.worldPos.y * _ScanlineFreq - _Time.y * _ScanlineSpeed);
scanline = scanline * 0.5 + 0.5;

fixed4 col = _Color;
col.a = _Color.a * rimIntensity * lerp(1.0, scanline, _ScanlineIntensity);
```

O flicker multiplica o alpha por uma onda de 40Hz quando o glitch está ativo, criando a instabilidade visual de uma projeção com interferência:

```
float flicker = lerp(1.0, sin(_Time.y * 40.0) * 0.5 + 0.5, _GlitchIntensity * glitchSnap);
col.a *= flicker;
```

3.5 Integração C# — HologramController

Em Start(), o script encontra o jogador pela tag "Player" e itera sobre todos os Renderer filhos com GetComponentInChildren<Renderer>(), criando uma instância única do material para cada um (new Material(materialHologramaBase)). Isto é essencial: se partilhassem o mesmo material, alterar _GlitchIntensity num objeto afetaria todos os outros.

Em Update(), calcula a distância ao jogador e usa Mathf.InverseLerp para mapear a distância para um valor normalizado de intensidade. Define também _IsConstantGlitch a 1 quando o jogador entra na zona constantGlitchDistance. Os property IDs são cacheados em Start() com Shader.PropertyToID para evitar pesquisas de string por frame — prática essencial em shaders dinâmicos:

```
glitchIntensityID = Shader.PropertyToID("_GlitchIntensity");
isConstantGlitchID = Shader.PropertyToID("_IsConstantGlitch");
```

O método OnDrawGizmosSelected desenha três esferas no editor (amarela, vermelha e magenta) que representam as três zonas de distância, permitindo ajustar os valores visualmente sem entrar em Play Mode.

3.6 Built-in Variables e Funções CG

`_Time` (—) → *float4*

Vetor de tempo global do Unity. $.x=t/20$, $.y=t$, $.z=t*2$, $.w=t*3$. Usado como `_Time.y` (segundos) para animar breathing, scanlines, glitch e flicker de forma contínua sem estado externo.

UnityObjectToWorldNormal (float3 normal) → *float3*

Transforma uma normal do espaço objeto para world space usando a transposta da inversa da matriz de modelo. Essencial com escalonamento não uniforme — multiplicar por `unity_ObjectToWorld` daria normais erradas com escala diferente em X, Y, Z.

UnityWorldSpaceViewDir (float3 worldPos) → *float3*

Retorna o vetor não normalizado do fragmento para a câmara em world space. Deve ser normalizado antes de usar em dot products. Equivalente a `(_WorldSpaceCameraPos - worldPos)`.

saturate (float/floatN x) → *mesmo tipo*

Clamp para $[0,1]$ sem branch. É uma instrução de hardware em GPUs modernas, mais eficiente que `clamp(x,0,1)` na maioria das arquiteturas.

step (float edge, float x) → *float*

Retorna 0 se $x < \text{edge}$, 1 caso contrário. Threshold binário sem branch, importante para performance em shaders que correm por fragmento.

max (float a, float b) → *float*

Máximo entre dois valores. Usado para implementar lógica OR sem branch: `max (baseSnap, _IsConstantGlitch) = 1` se qualquer um for 1.

pow (float base, float exp) → *float*

Potência. Controla a curvatura do rim: valores altos concentram o brilho numa linha fina nas bordas, valores baixos espalham-no por toda a face.

lerp (float a, float b, float t) → *float/floatN*

Interpolação linear: $a + (b-a) * t$. Usada para misturar scanlines no alpha e para o flicker.

4. Energy Shield

4.1 Descrição e Intenção

O EnergyShield simula o escudo de plasma de uma instalação científica. Quando um objeto colide com o escudo, uma onda de impacto expande-se radialmente a partir do ponto de contacto, deformando a geometria ao longo do caminho e alterando a cor na zona afetada. A escolha de um geometry shader foi intencional: o efeito de expansão geométrica precisa de deslocar vértices de forma coordenada por face, algo impossível num fragment shader (que opera por pixel, sem acesso à geometria vizinha) e insuficiente num vertex shader isolado (que não tem visão da face como unidade — cada vértice é processado independentemente, sem conhecimento dos seus vizinhos imediatos).

4.2 Pipeline

Um único passe com três stages: vertex, geometry e fragment. A diretiva `#pragma target 4.0` é obrigatória para ativar o geometry shader. O pipeline usa duas structs de passagem intermédia: `v2g` entre o vertex e o geometry shader, e `g2f` entre o geometry e o fragment. Esta separação é necessária porque o geometry shader acrescenta `viewDir`, que só pode ser calculado depois de ter acesso à posição world de cada vértice. O render state configura transparência standard:

```
Tags{"RenderType" = "Transparent" "Queue" = "Transparent+5"}  
Blend SrcAlpha OneMinusSrcAlpha  
ZWrite Off  
Cull Back
```

`"Queue=Transparent+5"` garante que o escudo renderiza depois de outros objetos transparentes na cena, evitando que fique tapado por eles. **ZWrite Off** impede que o escudo escreva no depth buffer, o que bloquearia a renderização de objetos atrás de si. **Cull Back** descarta as faces traseiras, o que é adequado para uma esfera vista de fora.

4.3 Vertex Shader

O vertex shader é um pass-through mínimo. Não transforma para clip space porque o geometry shader precisa de receber os dados ainda em object space para modificar a geometria antes da transformação final:

```
v2g vert(appdata v)
{
    v2g o;
    o.vertex = v.vertex;
    o.worldPos = mul(unity_ObjectToWorld, v.vertex).xyz;
    o.normal = UnityObjectToWorldNormal(v.normal);
    return o;
}
```

4.4 Geometry Shader — Ripple Geométrico

O geometry shader recebe um triângulo completo (3 vértices) e tem visão global da face, o que é precisamente a razão da sua escolha. A decisão de deslocar é tomada ao nível da face e não do vértice, evitando deformações irregulares causadas por vértices partilhados entre faces adjacentes.

O centro do triângulo é calculado como a média das posições dos três vértices, e a normal da face como a média das suas normais. A onda expande-se radialmente: o raio atual é `_ImpactTime * _MaxRadius`. O `smoothstep` com os parâmetros invertidos (`edge0 = _RippleWidth`, `edge1 = 0.0`) cria uma janela suave centrada na frente de onda. O `(1 - _ImpactTime)` faz a amplitude diminuir à medida que a onda se expande, simulando dissipação de energia:

```
float3 center = (IN[0].worldPos + IN[1].worldPos + IN[2].worldPos) / 3.0;
float3 faceNormal = normalize(IN[0].normal + IN[1].normal + IN[2].normal);
float currentRadius = _ImpactTime * _MaxRadius;
float dist = length(center - _ImpactPos.xyz);
float onWave = smoothstep(_RippleWidth, 0.0, abs(dist - currentRadius));
float active = step(0.001, _ImpactTime);
float expand = onWave * 0.04 * (1.0 - _ImpactTime) * active;
```

O `step(0.001, _ImpactTime)` serve como guarda: quando não há impacto ativo (`_ImpactTime == 0`), `active` é 0 e nenhum deslocamento ocorre, evitando triângulos levantados desnecessariamente.

Cada vértice é então deslocado ao longo da normal da face, reconvertido de world space para object space, e finalmente transformado para clip space. Este percurso é necessário porque o

deslocamento é feito em world space (onde as distâncias ao ponto de impacto fazem sentido), mas **UnityObjectToClipPos** espera object space:

```
float3 newWorldPos = IN[i].worldPos + faceNormal * expand;
float4 objPos = mul(unity_WorldToObject, float4(newWorldPos, 1.0));
g2f o;
```

4.5 Fragment Shader — Fresnel e Ripple de Cor

O fragment shader calcula dois efeitos visuais independentes que são depois compostos.

O efeito de Fresnel produz o brilho característico nas bordas do escudo. Quando a superfície está perpendicular ao olhar (bordas da esfera), $\text{dot}(\mathbf{N}, \mathbf{V}) \approx 0$, o que eleva o valor de Fresnel e aumenta a opacidade e brilho nessas zonas. `_FresnelEffect` controla a dureza da transição — valores altos confinam o brilho às extremidades mais oblíquas:

```
float fresnel = pow(1.0 - saturate(dot(N, V)), _FresnelEffect);
```

O ripple de cor usa a mesma lógica de distância do geometry shader, mas calculado por fragment para garantir uma transição suave independente da resolução da malha. `rippleFade` inclui o fator de desvanecimento temporal, de modo que a cor da onda desapareça progressivamente:

```
float rippleFade = ripple * (1.0 - _ImpactTime) * step(0.001, _ImpactTime);
```

A composição final mistura os dois efeitos sobre a cor base. O lerp substitui a cor base pela cor da onda proporcionalmente a `rippleFade`, e o alpha acumula três contribuições independentes.

O `saturate` no retorno garante que a soma de múltiplos contributos não ultrapassa 1:

```
col.rgb = lerp(col.rgb, _RippleColor.rgb, rippleFade);
col.a = _Color.a + fresnel * 0.3 + rippleFade * 0.5;
return saturate(col);
```

4.6 Integração C# — EnergyShieldController

O EnergyShieldController instância uma cópia privada do material em Start() com shieldRenderer.material. Os property IDs são guardados em cache como static readonly com Shader.PropertyToID, de modo a existirem uma única vez em memória independentemente do número de instâncias do script.

OnCollisionEnter captura o primeiro ponto de contacto da colisão e passa-o a TriggerImpact() em world space. OnTriggerEnter usa Collider.ClosestPoint() para encontrar o ponto da superfície do collider mais próximo do objeto que entrou, obtendo assim um ponto de impacto mais preciso do que simplesmente usar a posição do objeto.

TriggerImpact() interrompe qualquer animação em curso com StopCoroutine antes de iniciar uma nova, evitando que dois impactos rápidos criem estados inconsistentes no material. A coroutine incrementa _ImpactTime de 0 a 1 ao longo de impactDuration segundos usando Time.deltaTime, e repõe o valor a 0 no fim para que o escudo volte ao estado base. OnDestroy() destrói a instância privada do material.

4.7 Built-in Variables e Funções CG

unity_ObjectToWorld / unity_WorldToObject (—) → *float4x4*

Matrizes de transformação objeto→mundo e mundo→objeto. No geometry shader usa-se unity_WorldToObject para reverter posições modificadas em world space de volta a object space antes de UnityObjectToClipPos.

smoothstep (float edge0, float edge1, float x) → *float*

Interpolação cúbica Hermite. Ao contrário de lerp, a derivada é 0 nos extremos, eliminando descontinuidades visíveis nas fronteiras da onda de impacto. Recebe os valores de edge invertidos (edge1 < edge0) para obter o efeito de janela na frente de onda.

length (floatN v) → *float*

Módulo euclidiano de um vetor. Calcula a distância do centro de cada face ao ponto de impacto em cada invocação do geometry shader.

normalize (floatN v) → *floatN*

Retorna o vetor unitário (comprimento 1). Obrigatório antes de qualquer dot product de iluminação: um vetor não normalizado distorce o resultado do Fresnel.

5. CCTV Glitch (Post-Process)

5.1 Descrição e Intenção

O CCTV_Glitch é um efeito de pós-processamento aplicado sobre o *render buffer* completo da câmara através dos métodos `OnRenderImage` e `Graphics.Blit`. Este shader não processa geometria 3D da cena; opera exclusivamente sobre a *Render Texture* da câmara como se fosse uma imagem 2D plana. O seu objetivo é simular uma câmara de vigilância danificada, com artefactos de compressão digital e interferência eletromagnética.

A intensidade do efeito é dinâmica e composta por duas variáveis geridas via C#: picos esporádicos orgânicos e um aumento contínuo baseado na proximidade do jogador a uma anomalia (o *Phone Microwave*). O shader utiliza o prefixo `Hidden/` na sua declaração para não ser listado no menu de shaders do Inspector, visto que o seu uso é exclusivo do script `CCTVGlitchEffect.cs`.

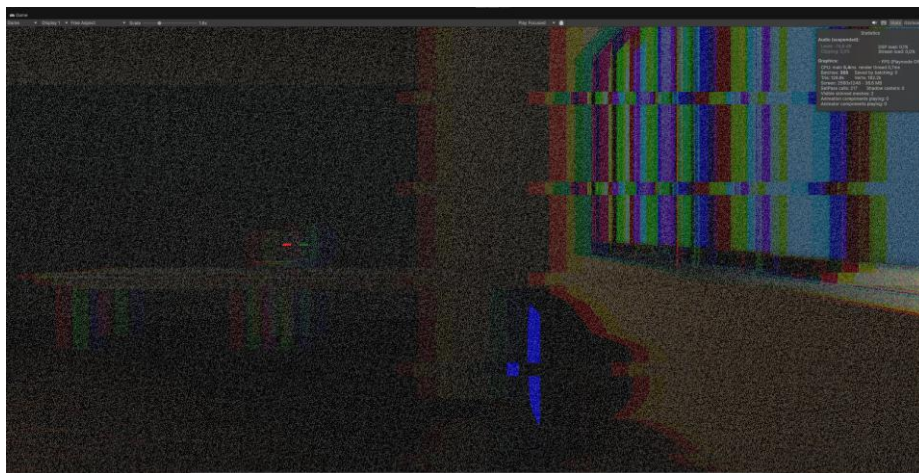


Figura — CCTV Glitch e ReadingSteiner ativos em simultâneo

5.2 Pipeline

O efeito é executado num único passe (*Pass*) com as *flags* Cull Off, ZWrite Off e ZTest Always. Em conjunto, estas três instruções garantem que o *quad* de pós-processamento (que cobre a totalidade do ecrã) ignora o *Z-Buffer* e nunca é descartado por testes de profundidade (*Depth Testing*). A textura principal (*_MainTex*) é a própria *Render Texture* da câmara, injetada via *Graphics.Blit*.

5.3 Função random — Conversão de GLSL

Para a geração de ruído estático, implementou-se uma função de *hash* pseudo-aleatória, adaptada de referências clássicas do repositório *Shadertoy*. As diferenças sintáticas entre GLSL e HLSL/Cg são mínimas, mas obrigatórias para a compilação no Unity:

GLSL (Shadertoy)	HLSL / Cg (Unity)
<pre>// GLSL (Shadertoy) float random (vec2 st) { return fract (sin (dot (st, vec2(12.9898, 78.233))) * 43758.5453123); }</pre>	<pre>// HLSL/Cg (Unity) float random (float2 st) { return frac (sin (dot (st, float2(12.9898, 78.233))) * 43758.5453123); }</pre>

A lógica matemática permanece idêntica: o produto escalar (*dot product*) com os valores irracionais 12.9898 e 78.233 combina as coordenadas X e Y num único valor escalar. O seno de um número multiplicado por uma constante elevada (43758.5453) oscila de forma caótica, e a função *frac* garante que o **resultado** é mantido no intervalo [0, 1). Este processo é determinístico, mas visualmente impercetível de um ruído aleatório.

5.4 Abertura Cromática

O shader simula falhas de sincronização vertical (*Tearing*) ao somar um desvio matemático ao eixo X das coordenadas UV, deslocando secções da imagem horizontalmente com base na intensidade do glitch.

Em simultâneo, aplica-se o efeito de **Aberração Cromática**, que simula a separação ótica de comprimentos de onda típica de lentes imperfeitas ou sinais de vídeo degradados. Cada canal de cor (RGB) é amostrado de uma coordenada UV diferente no eixo horizontal. Este deslocamento é diretamente proporcional à variável `_GlitchIntensity`:

```
float4 colR = tex2D(_MainTex, uv - float2(_ChromAberration * _GlitchIntensity, 0));
float4 colG = tex2D(_MainTex, uv);
float4 colB = tex2D(_MainTex, uv + float2(_ChromAberration * _GlitchIntensity, 0));
float4 col = float4(colR.r, colG.g, colB.b, 1.0);
```

5.5 Scanlines, Grão e Desaturação

O efeito de ruído televisivo divide-se em três partes:

- **Scanlines:** Utilizam a função `sin ()` multiplicada pela coordenada Y e animada com `_Time.y`. O resultado é subtraído à cor base, escurecendo linhas horizontais alternadas de forma rítmica.
- **Grão (Film Grain):** Recorre à função `random ()`, descrita anteriormente, centrando o ruído no valor de zero (`noise - 0.5f`) e somando-o aos píxeis.
- **Desaturação:** Para simular as limitações da transmissão CCTV analógica, a cor perde saturação com base na equação de luminância BT.601, utilizando os pesos percetuais do olho humano: $Y=0.299R+0.587G+0.114B$. A cor original é interpolada (`lerp`) com esta escala de cinzentos a 40%.

5.6 Integração C# — CCTVGlitchEffect

No `Update()`, o script combina dois cálculos de intensidade:

Glitch Orgânico: Um temporizador entre 0.1 e 1.0 segundos que dispara picos imprevisíveis com 15% de probabilidade, atingindo valores entre 0.4 e 0.8 de intensidade. Este salto é suavizado por um `Mathf.Lerp`.

Proximidade Espacial: Calcula a distância em metros até à anomalia. Usa o método `Mathf.Clamp01` para converter a distância mínima e máxima numa percentagem linear de interferência.

Ambos os valores são somados e limitados ao valor máximo de 1.0, injetando o resultado na variável `_GlitchIntensity` do shader a cada *frame*.

5.7 Built-in Variables e Funções CG

tex2D (sampler2D s, float2 uv) → *fixed4*

Amostra uma textura 2D nas coordenadas UV dadas. No contexto de pós-processamento, `_MainTex` é o render texture da câmara passado pelo `Graphics.Blit`. Cada canal pode ser amostrado de UVs diferentes para criar a abertura cromática.

frac (float/floatN x) → *mesmo tipo*

Parte fracionária de x ($x - \text{floor}(x)$). Em GLSL chama-se `fract()`. Garante output em $[0, 1)$. Usada na função `random`.

dot (floatN a, floatN b) → *float*

Produto escalar: $\sum(a[i] * b[i])$. Usado no hash para combinar as duas coordenadas UV num escalar, e para o cálculo de luminosidade BT.601 (dot com os pesos de cada canal).

6. Reading Steiner (Post-Process)

6.1 Descrição e Intenção

Reading Steiner é a capacidade de Rintarou Okabe de reter memórias ao mudar de linha de universo. O shader traduz essa experiência: quando o jogador ativa a mudança de linha, o ecrã sofre distúrbios crescentes — screen tearing, separação RGB e ruído — até um clímax onde a cor inverte, representando a percepção distorcida da transição. A escolha de post-process é intencional: o efeito deve afetar toda a percepção do jogador, não apenas um objeto da cena.

Tal como o CCTV, usa Hidden/ no nome e a mesma estrutura de passe (Cull Off, ZWrite Off, ZTest Always).

6.2 Screen Tearing

O step sobre seno de alta frequência em Y cria faixas horizontais binárias (0 ou 1) que deslocam secções do ecrã horizontalmente, simulando o artefacto de tearing de uma cassete VHS ou sinal de vídeo perturbado. A frequência 50 e velocidade 20 foram calibradas para criar faixas largas que se movem rapidamente:

```
float tearBand = step(0.9, sin(uv.y * tearFrequency + _Time.y * tearSpeed));  
float tearOffset = tearBand * (_GlitchIntensity * 0.1);  
uv.x += tearOffset;
```

6.3 RGB Split e Ruído

A separação cromática é mais agressiva que no CCTV (splitAmount = _GlitchIntensity * 0.08 vs 0.01-0.05) porque o ReadingSteiner é um evento de clímax dramático. O canal R é deslocado para a direita e o B para a esquerda, criando a separação bicromática. O ruído aditivo centrado em zero é escalado por _GlitchIntensity para crescer com a intensidade.

6.4 Inversão de Cor

Na fase de clímax (`_GlitchIntensity >= 0.9`), pixels invertem-se aleatoriamente. O `_Time.x` (`= _Time.y / 20`) garante que a aleatoriedade muda mais lentamente que por frame, criando flashes em vez de flickering de pixel por pixel. O `step(0.5, rand(...))` decide se aquele pixel inverte ou não, resultando em aproximadamente 50% dos pixels invertidos na fase de clímax:

```
float invertTrigger = step(0.9, _GlitchIntensity);
float3 finalColor = float3(r, g, b) + noise;

finalColor = lerp(finalColor, 1.0 - finalColor, invertTrigger * step(0.5, rand(uv * _Time.x)));
```

6.5 Integração C# — ReadingSteinerEffect e MicrowaveReadingSteinerDriver

O `ReadingSteinerEffect.cs` expõe `currentIntensity` como public float. O `OnRenderImage` só ativa o `Graphics.Blit` se `currentIntensity > 0.01f`, evitando uma cópia desnecessária do render buffer quando o efeito não está ativo. A Coroutine interna `ShiftRoutine` usa rampa cúbica (`percent^3`) para aceleração exponencial.

O `MicrowaveReadingSteinerDriver.cs` orquestra toda a sequência narrativa. A condição de disparo exige que o jogador esteja a segurar a pen (`isHeld = true` via `selectEntered/selectExited`) E que o microondas esteja ligado (`microwaveSwitch.IsMicrowaveOn`), verificado em `OnActivated`. A flag `hasLeaped` torna o salto irrepetível — uma vez feito, o `shiftCoroutine` permanece não-null permanentemente como mecanismo de lock.

A Coroutine `WorldlineShiftSequence` corre em 6 fases sequenciais: (1) rampa cúbica de 2.5s até `maxIntensity=1.2`; (2) hold de 0.4s no clímax; (3) teleporte para a Sala dos Clones, ativa `cloneManagerRoot` e `cctvEffect`; (4) espera de 15s; (5) pico instantâneo de 0.2s; (6) teleporte de regresso, desativa clones, altera objetos da cena e injeta `HologramController` dinamicamente com `AddComponent` nos objetos de `paisParaHolograma`. O teleporte usa `CharacterController.enabled = false` antes de mover o jogador para evitar que o CC resistisse ao movimento forçado.

6.6 Built-in Variables e Funções CG

`_Time.x` (—) $\rightarrow$ *float*

`_Time.y` / 20. Gera aleatoriedade que muda 20x mais lentamente que `_Time.y`. Ao usar `rand(uv * _Time.x)`, o padrão de inversão muda a cada ~ 0.05 segundos em vez de cada frame, criando flashes perceptíveis em vez de ruído por pixel.

7. Kerr — Buraco Negro Volumétrico

7.1 Descrição e Intenção

O módulo Kerr é um motor de simulação ótica não-linear executado inteiramente ao nível do *Fragment Shader* através de um algoritmo de **Raymarching Volumétrico**. O subsistema simula computacionalmente a métrica de Kerr para buracos negros rotativos, equacionando simultaneamente três fenômenos da astrofísica relativística: a deflexão espaço-temporal da luz (*gravitational lensing*), o perfil de densidade de um disco de acreção equatorial, e a anisotropia luminosa induzida pelo **Efeito Doppler Relativístico**.

A abordagem por geometria explícita (*rasterization* tradicional) foi imediatamente descartada devido à impossibilidade matemática de curvar primitivas de polígonos de forma contínua em tempo real. O pipeline foi, por isso, desenhado como um sistema híbrido: uma esfera unitária delimita o volume cartesiano de atuação da gravidade, delegando ao *GPU thread pipeline* o cálculo iterativo do percurso de cada fotão.



Figura — *Shader Kerr dentro do microondas, com disco de acreção e lensing gravitacional*

7.2 Otimização do Pipeline e Isolamento de Espaço de Cálculo

Para eliminar a sobrecarga computacional de uma câmara secundária e o subsequente custo de preenchimento (*fill-rate*) associado a *Render Textures*, a infraestrutura utiliza um comando nativo **GrabPass { "_BackgroundTexture" }**.

Esta instrução força o ROP (*Render Output Unit*) da GPU a clonar o *frame buffer* opaque atual diretamente na VRAM. Para contrariar o impacto desta operação na taxa de atualização do headset VR, o ciclo de vida deste buffer é gerido dinamicamente via C# (*BlackHoleActivationButton.cs*), garantindo que a alocação de memória e a paragem do pipeline de desenho ocorram exclusivamente quando a anomalia está ativa.

Ao nível do *Vertex Shader*, a computação espacial foi otimizada para mitigar operações matriciais repetitivas por fragmento:

```

v2f vert (appdata v)
{
    v2f o;
    o.vertex = UnityObjectToClipPos(v.vertex);
    o.screenPos = ComputeGrabScreenPos(o.vertex);
    o.localPos = v.vertex.xyz;
    o.localCamPos = mul(unity_WorldToObject, float4(_WorldSpaceCameraPos, 1.0)).xyz;
    return o;
}

```

Ao projetar o vetor da câmara global (`_WorldSpaceCameraPos`) para o espaço local do objeto através da matriz inversa `unity_WorldToObject`, convertemos a matriz de cálculo do Raymarcher numa esfera unitária perfeita centrada na origem (0,0,0). Esta simplificação geométrica converte operações dispendiosas de distância em coordenadas globais para simples computações de magnitude vetorial local, poupando centenas de instruções aritméticas por ciclo de *thread*.

7.3 Motor de Raymarching e Integração Vetorial Gravitacional

O loop principal de amostragem executa um integrador numérico limitado por um teto rígido de iterações (`_MaxSteps`), mitigando cenários de divergência de execução (*execution branch divergence*) nas unidades SIMD da GPU:

```

for (int step = 0; step < _MaxSteps; step++)
{
    float distFromCenter = length(rayPos);

    if (distFromCenter < _EventHorizon)
    {
        hitHorizon = true;
        break;
    }

    if (distFromCenter > 0.51) break;

    float3 gravityPull = -normalize(rayPos) * (_Mass / (distFromCenter * distFromCenter + 0.001));
    rayDir = normalize(rayDir + gravityPull * _StepSize);
}

```

O percurso do raio simula a aceleração fotónica através de um cálculo pseudo-newtoniano diferencial. A cada passo, calcula-se o vetor de atração central (`-normalize(rayPos)`) escalado pelo inverso do quadrado da distância. O fator de segurança 0.001 foi injetado para garantir a estabilidade numérica do pipeline, prevenindo exceções de divisão por zero na singularidade.

Para otimizar o rendimento gráfico, foram implementadas duas cláusulas de saída antecipada (break):

- **Colisão Absoluta:** O raio penetra o horizonte de eventos e é imediatamente descartado.
- **Escape de Fronteira:** O raio ultrapassa o limite de influência gravitacional (0.51), terminando a execução da *thread* de forma assíncrona e libertando imediatamente os recursos de computação do núcleo da GPU.

7.4 Disco de Acrensão

O disco de acreção é processado como uma densidade gasosa volumétrica sem superfícies rígidas, mapeado puramente por restrições analíticas no plano equatorial local ($|y| < \text{espessura}$). A geração de turbulência e aglomerados de matéria funde duas frequências sinusoidais cruzadas baseadas em coordenadas polares (atan2).

```
float angle = atan2(rayPos.z, rayPos.x);
float spin = angle + _Time.y * _SpinSpeed;

float radialNoise = sin(distFromCenter * 40.0) * 0.5 + 0.5;
float angularNoise = sin(spin * 6.0) * 0.5 + 0.5;
float ringNoise = radialNoise * angularNoise;

float3 spinDir = normalize(cross(float3(0, 1, 0), rayPos));
float doppler = pow(max(0.0, dot(rayDir, spinDir)), 2.0);

float stepGlow = verticalDensity * radialFalloff * ringNoise * _DiskDensity * _StepSize;

accumulatedGlow.rgb += _DiskColor.rgb * stepGlow * (doppler * 3.0 + 0.2);
accumulatedGlow.a += stepGlow;
```

A assimetria visual crítica da simulação baseia-se no cálculo vetorial do Efeito Doppler. Através do produto vetorial entre o vetor vertical da anomalia e a posição do fotão, determinamos o vetor tangente orbital instantâneo do gás (spinDir). O produto escalar (dot) avalia a projeção cinemática: se a matéria se desloca contra a direção do raio (aproximando-se do observador), o valor de emissão sofre uma amplificação exponencial de magnitude. Isto traduz-se fielmente na assinatura visual real observada em astrofotografia de alta resolução.

7.5 Output Final e Distorção do Fundo

No término do percurso do raio, se este sobreviveu à atração gravitacional, o desvio angular sofrido é extraído subtraindo o vetor de direção final ao inicial.

```
float2 distortion = (rayDir.xy - initialRayDir.xy) * _WarpStrength;  
float2 lensedUV = originalScreenUV + distortion;  
  
float4 background = tex2D(_BackgroundTexture, lensedUV);  
return background + accumulatedGlow;
```

Este diferencial vetorial 3D é convertido num vetor de deslocamento 2D no espaço de ecrã (*Screen Space UV*). Através de uma divisão de perspetiva homogénea ($i.screenPos.xy / i.screenPos.w$), o shader deforma a amostragem da `_BackgroundTexture` capturada previamente. O resultado é uma lente ótica dinâmica de alta precisão que distorce o cenário do laboratório em tempo real sem qualquer dependência de malhas poligonais complexas, garantindo fidelidade visual e imersão absoluta em ambientes de Realidade Virtual.

7.6 Integração C# — BlackHoleActivationButton e SpinUp

`BlackHoleActivationButton.cs` usa `selectEntered` do `XRBaseInteractable`: ao pressionar o botão físico do microondas, ativa o `GameObject` do buraco negro (desativado por defeito para não ter custo de `GrabPass` quando não é necessário) e chama `KerrAnomaly.ActivateAnomaly()`. Expõe `IsMicrowaveOn` como propriedade pública para o `MicrowaveReadingSteinerDriver` verificar. A flag `disableButtonAfterUse` interrompe o interactor após o primeiro uso.

`SpinUp.cs` (`KerrAnomaly`) anima a formação do buraco negro com rampa quadrática ao longo de 3 segundos, aumentando progressivamente `_Mass` e `_SpinSpeed` via `SetFloat` no material.

7.7 Built-in Variables e Funções CG

ComputeGrabScreenPos (float4 clipPos) → *float4*

Converte uma posição em clip space para coordenadas UV do GrabPass. Trata a inversão da coordenada Y em Direct3D vs OpenGL. Sem esta função, `_BackgroundTexture` ficaria vertically flipped em algumas plataformas.

unity_WorldToObject (—) → *float4x4*

Inversa de `unity_ObjectToWorld`. Usada no vertex shader para transformar `_WorldSpaceCameraPos` (world space) para object space do buraco negro, permitindo que o raymarcher opere em espaço local.

_WorldSpaceCameraPos (—) → *float3*

Posição da câmara em world space. Fornecida automaticamente pelo Unity em cada frame. Usada para calcular a origem do raymarcher em object space.

atan2 (float y, float x) → *float*

Arco-tangente de dois argumentos, retornando o ângulo no intervalo $[-\pi, \pi]$. Ao contrário de `atan(y/x)`, trata todos os quadrantes e a divisão por zero. Calcula o ângulo azimuth completo de 360° do disco.

cross (float3 a, float3 b) → *float3*

Produto vetorial. Retorna um vetor perpendicular a ambos os inputs, com comprimento $|a| |b| \sin(\text{ang})$. `cross(up, radial)` dá a tangente orbital — a direção em que o gás se move no disco.

8. Radioactive Liquid

8.1 Descrição e Intenção

O RadioactiveLiquid simula líquido radioativo dentro de um recipiente de vidro. A técnica de stencil buffer multi-pass foi escolhida porque permite usar a própria geometria do frasco como máscara de renderização, funcionando automaticamente em qualquer forma ou tamanho sem ajustes manuais. As alternativas — uma mesh plana posicionada manualmente, clip planes matemáticos ou render texture com câmara secundária — não escalariam com a diversidade de formas dos frascos na cena.

Não existe script C# associado a este shader. Toda a animação corre no próprio shader via `_Time.y`. Os parâmetros são ajustados manualmente no Inspector.



Figura — Radioactive Liquid em frascos de vários tamanhos

8.2 Arquitetura Multi-Pass com Stencil

A SubShader tem `Queue=Transparent` e três passes que correm sempre por esta ordem:

- **Pass Glass** (`Cull Back`, `Blend transparente`, `ZWrite Off`): renderiza o exterior do vidro com Fresnel. As bordas ficam mais opacas que o centro, simulando o comportamento óptico real do vidro.
- **Pass StencilWrite** (`Cull Front`, `ColorMask 0`, `ZWrite Off`, `Stencil Comp Always` `Pass Replace Ref 1`): renderiza o back-face do frasco sem pintar nenhum pixel, escrevendo 1 no stencil buffer nos pixels correspondentes ao interior. `Cull Front` é essencial: renderiza as faces interiores da malha, que correspondem ao volume interno do frasco.
- **Pass Liquid** (`Cull Off`, `Blend transparente`, `ZWrite Off`, `Stencil Comp Equal Ref 1`): renderiza o líquido apenas onde o stencil é 1. `Cull Off` para ser visível de qualquer ângulo.

8.3 Vertex Shader (Pass Liquid)

O vertex shader calcula apenas worldPos em world space: mul(unity_ObjectToWorld, v.vertex).xyz. É fundamental que as ondas e o clip sejam calculadas em world space para que a superfície do líquido permaneça horizontal no mundo independentemente da rotação do frasco, comportando-se como líquido real.

8.4 Smooth Noise — Conversão de GLSL

A função smoothNoise foi adaptada de Value Noise em GLSL do Shadertoy. As diferenças são sintáticas:

GLSL (Shadertoy)	HLSL / Cg (Unity)
<pre>// GLSL float hash(vec2 p) { return fract(sin(dot(p, vec2(127.1, 311.7))) * 43758.5453); } float smoothNoise(vec2 p) { vec2 i = floor(p); vec2 f = fract(p); float a = hash(i); float b = hash(i+vec2(1,0));</pre>	<pre>// HLSL/Cg (Unity) float hash(float2 p) { return frac(sin(dot(p, float2(127.1, 311.7))) * 43758.5453); } float smoothNoise(float2 p) { float2 i = floor(p); float2 f = frac(p); float a = hash(i); float b = hash(i+float2(1,0));</pre>

<pre>float c = hash(i+vec2(0,1)); float d = hash(i+vec2(1,1)); vec2 u = f*f*(3.0-2.0*f); return mix(mix(a,b,u.x), mix(c,d,u.x),u.y); }</pre>	<pre>float c = hash(i+float2(0,1)); float d = hash(i+float2(1,1)); float2 u = f*f*(3.0-2.0*f); return lerp(lerp(a,b,u.x), lerp(c,d,u.x),u.y); }</pre>
---	--

Diferenças: vec2→float2, fract→frac, mix→lerp. A lógica é idêntica: grid de células com hash nos cantos, interpolação bilinear com o polinômio cúbico $f^2(3-2f)$. Este polinômio tem derivada 0 nas bordas das células, eliminando as arestas visíveis que ocorreriam com lerp linear simples.

8.5 Fragment Shader — Ondas, Clip, Espuma e Glow

O fragment shader corre em sequência lógica para cada pixel que passou o teste de stencil:

Primeiro calcula as 4 ondas: primaryWave (sin em X, onda base), secondaryWave (cos em Z, direção perpendicular com velocidade 1.3x), turbulence (sin*cos, cria picos e vales irregulares), e noiseWave (smoothNoise a $_Time.y*0.5$, nunca repete). A soma das quatro cria movimento orgânico sem loop perceptível.

De seguida calcula a posição da superfície. `unity_ObjectToWorld[1][3]` é a componente Y da coluna de tradução da matriz 4x4 — a posição Y do centro do objeto em world space sem preciso de um `mul()` completo. O `objectRadius` extrai a escala Y pelo comprimento da coluna Y de rotação/escala:

```
float objectCenterY = unity_ObjectToWorld[1][3];
float objectRadius = length(float3(
    unity_ObjectToWorld[0][1],
    unity_ObjectToWorld[1][1],
    unity_ObjectToWorld[2][1]));

float surfaceWorldY = objectCenterY
    + (_FillLevel * objectRadius)
    + totalWave;

clip(surfaceWorldY - i.worldPos.y);
```

O `clip()` descarta fragmentos acima da superfície numa única instrução de hardware. É mais eficiente que `if/discard` porque não quebra o fluxo de execução paralela da GPU.

A espuma é a zona imediatamente abaixo da superfície: `1-smoothstep(0, _FoamThreshold, belowSurface)` dá 1 na superfície e vai a 0 abaixo do threshold. A transparência varia com a profundidade: `lerp(0.55, _Color.a, saturate(belowSurface*8))` — mais transparente na superfície, mais opaco no fundo. O glow usa `sin` animado com `totalWave*50` para que o pulso varie com a forma da onda, criando borbulhamento orgânico em vez de uma luz a piscar uniformemente.

8.6 Built-in Variables e Funções CG

clip (float x) → *void*

Descarta o fragmento (equivalente a `discard`) se $x < 0$. Instrução de hardware, mais eficiente que `if()` com `discard` porque não cria divergencia no warp de execução da GPU.

unity_ObjectToWorld[row][col] (—) → *float*

Acesso direto a uma componente individual da matriz. `[1][3]` é Y da tradução (coluna 3). Mais eficiente que um `mul()` completo para extrair um único valor escalar.

floor (float/floatN x) → *mesmo tipo*

Arredondamento para baixo. No `smoothNoise`, separa as coordenadas inteiras (identifica a célula do grid) das fracionárias (posição local dentro da célula).

smoothstep (float edge0, float edge1, float x) → *float*

Interpolação cúbica com derivada 0 nos extremos. Usado tanto na espuma (transição suave da superfície) como no `smoothNoise` (elimina arestas nas fronteiras das células).

9. Stencil Portal e Stencil World (X-Ray)

9.1 Descrição e Intenção

O par de shaders *StencilPortal* e *StencilWorld* atua em conjunto para implementar uma mecânica de portal de visão de raio-X através de superfícies sólidas. O portal (aplicado a um *quad* na parede) atua como um carimbo invisível que escreve uma máscara no *Stencil Buffer*. Em contrapartida, o objeto oculto atrás da parede apenas é renderizado nos píxeis onde essa máscara existe. A utilização da diretiva *ZTest Always* garante que o objeto oculto ignora o *Depth Buffer*, tornando-se visível através da geometria opaca.

A escolha do *Stencil Buffer* em detrimento de uma abordagem com uma câmara secundária e uma *Render Texture* deve-se à extrema eficiência deste método: não exige renderizar a geometria da cena duas vezes, limitando-se a realizar operações leves de marcação e leitura de valores inteiros por píxel.

Utilizou-se um modelo primitivo do tipo *Quad* para o portal porque o cálculo matemático da borda brilhante pressupõe um mapeamento UV de 0 a 1, perfeitamente centrado em (0.5, 0.5). O *Quad* possui este mapeamento por defeito, ao passo que uma esfera ou uma *mesh* irregular apresentaria distorções nas coordenadas UV, deformando o efeito visual.



Figura — Portal e Xray shader

9.2 StencilPortal — Shader "Custom/Stencil/PortalMask_Hologram"

A ordem de renderização é gerida através das *Tags*. A declaração `Queue=Geometry+1` garante que o portal é desenhado imediatamente após o cenário opaco, mas antes do objeto oculto (`Geometry+2`). O passe inclui `Blend SrcAlpha OneMinusSrcAlpha` para transparência, `ZWrite Off` para não obstruir objetos reais e `Cull Off` para renderizar ambas as faces.

```
Blend SrcAlpha OneMinusSrcAlpha
ZWrite Off
Cull Off

Stencil
{
    Ref 1
    Comp Always
    Pass Replace
}
```

Esta configuração força a escrita do valor 1 em todos os píxeis ocupados pelo *quad*.

O *Fragment Shader* calcula o brilho (*glow*) da borda com base na distância da coordenada UV ao centro. A distância em cada eixo é calculada algebricamente como $d = |uv - 0.5| \times 2.0$, mapeando o espaço UV de forma a ter o valor 0 no centro e 1 nas extremidades. A função `max` seleciona a borda mais próxima, e a função exponencial (`pow`) controla a atenuação da luz, criando um buraco transparente no centro com as extremidades brilhantes:

```
float2 dist = abs(i.uv - 0.5) * 2.0;

float edge = max(dist.x, dist.y);

float glow = pow(edge, _Thickness);

fixed4 finalColor = _BorderColor;
finalColor.a = glow;

return finalColor;
```

9.3 StencilWorld — Shader "Custom/Stencil/HiddenObject_XRay"

Este material encontra-se na fila de renderização subsequente (`Queue=Geometry+2`). A diretiva `ZTest Always` instrui a GPU a ignorar os testes de profundidade (*Z-Buffer*), forçando a sua renderização mesmo que exista uma parede física opaca entre o objeto e a câmara. A leitura do portal é feita no bloco `Stencil` com `Comp Equal` e `Ref 1`, garantindo que o objeto só existe dentro da silhueta do portal.

No *Vertex Shader*, a normal em *World Space* é calculada explicitamente através da multiplicação da matriz de transformação: $\mathbf{n}_{\text{world}} = \mathbf{M}_{\text{ObjectToWorld}} \times \mathbf{n}_{\text{local}}$.

No *Fragment Shader*, aplica-se o efeito de Raio-X através de uma equação de **Fresnel** otimizada, iluminando os contornos da geometria. A intensidade do Fresnel é inversamente proporcional ao produto escalar entre o vetor normal ($\mathbf{n}$) e o vetor de visão ($\mathbf{v}$): $F = (1 - \text{sat}(\mathbf{n} \cdot \mathbf{v}))^2$.

Adicionalmente, injetam-se *scanlines* animadas utilizando a função seno sobre o eixo Y local (objPos.y). Como o cálculo ocorre em *Object Space* e não em *World Space*, as linhas digitais rodam juntamente com o modelo 3D, conferindo volume e coerência espacial ao holograma:

```
float scanline = sin(i.objPos.y * 50.0 + _Time.y * _ScanSpeed);
scanline = saturate(scanline * 0.5 + 0.5);

fixed4 finalColor = _ScanColor * fresnel;
finalColor.rgb += _ScanColor.rgb * scanline * 0.5;
```

9.4 Built-in Variables e Funções CG

`WorldSpaceViewDir` (float4 vertex): Função nativa do Unity que recebe as coordenadas do vértice em *Object Space* e retorna o vetor direcional (não normalizado) que aponta do vértice até à câmara, já convertido para *World Space*. Difere da função homónima `UnityWorldSpaceViewDir` (que exige a posição previamente convertida para *World Space*). Este vetor deve ser sempre normalizado utilizando `normalize()` antes de ser utilizado em cálculos de iluminação geométrica, como é o caso do *dot product* do Fresnel.

10. Técnicas Consideradas e Não Implementadas

10.1 Interior Mapping

A técnica simula interiores de edifícios sem geometria adicional, usando ray-box intersection no fragment para calcular qual parede/chão/teto o raio encontra. Foi explorada conceitualmente, mas não integrada. Seria interessante para a cena do laboratório, mas exigiria conhecimento prévio das dimensões dos quartos em tempo de compilação, o que tornava a integração com a cena VR existente complexa.

10.2 Triplanar Mapping

Aplica texturas sem coordenadas UV usando as normais para misturar três projeções (XY, XZ, YZ). Considerado para o chão do laboratório, mas a cena usa assets com UVs próprias, tornando a técnica desnecessária.

10.3 Depth Texture

A leitura de `_CameraDepthTexture` foi explorada para criar soft particles no `RadioactiveLiquid`, onde o líquido se mistura suavemente com os objetos que toca. Não foi implementada por exigir, ativar `depthTextureMode` na câmara por C# e um passe extra, o que complicava a arquitetura multi-pass já existente.

10.4 Hull Shader (Tesselação)

Um hull shader para tesselação dinâmica do `EnergyShield` permitiria subdividir a malha adaptativamente perto do ponto de impacto. Descartado por compatibilidade: `#pragma target 4.6` não funciona em hardware móvel ou WebGL. O geometry shader existente cobre o requisito de diversidade de shader types.

11. Dificuldades e Soluções

11.1 Geometry Shader e #pragma target 4.0

O EnergyShield exigiu #pragma target 4.0. Em hardware anterior a DirectX 10 é incompatível.

Solução: FallBack Off com o projeto limitado a plataforma Desktop.

11.2 Ordem dos Passes no RadioactiveLiquid

O problema inicial era o líquido aparecer fora do frasco. O StencilWrite usava Cull Back por defeito, marcando o exterior. A solução foi Cull Front, que renderiza o back-face (interior do frasco), garantindo que só o volume interno é marcado.

11.3 GrabPass e Compatibilidade

O GrabPass do Kerr não é compatível com URP/HDRP e é ineficiente em móvel. A decisão de manter na BiRP foi consciente.

11.4 Conversões GLSL → HLSL

As funções random e smoothNoise foram adaptadas de GLSL. As substituições (fract→frac, mix→lerp, vec2→float2) foram verificadas visualmente comparando outputs em Shadertoy com a cena Unity.

11.5 Integração XR e Cadeia de Scripts

A cadeia de interações VR exigiu coordenação entre vários scripts. O padrão adotado foi: BlackHoleActivationButton gere o estado global do micro-ondas, MicrowaveReadingSteinerDriver verifica esse estado antes de disparar. A injeção dinâmica do HologramController com AddComponent evita Update() desnecessário antes do salto temporal.

11.6 CharacterController e Teleporte

O CharacterController do jogador resiste a posições forçadas, cancelando o teleporte. A solução foi desativar o CC (cc.enabled = false), mover o jogador, e reativar imediatamente. Esta ordem é crítica.

12. Portal Dimensional — The Casting of Frank Stone (Magic Circle)

12.1 Identificação do Efeito no Jogo

The Casting of Frank Stone (Behaviour Interactive, 2024) é um jogo de terror narrativo inserido no universo de *Dead by Daylight*. Durante a campanha, o jogador depara-se com um portal dimensional que surge como uma abertura retangular — reutilizando a forma de uma porta — que serve de passagem para uma dimensão alternativa dominada pela Entity. O efeito visual do portal é composto por cinco camadas sobrepostas, cada uma com uma função distinta.

12.1.1 Camadas Visuais Identificadas

Borda com Fresnel: As arestas da porta emitem luz verde intensa. O centro do portal é semitransparente enquanto as bordas são quase sólidas, criando o comportamento típico do efeito Fresnel — maior refletividade em ângulos rasantes à superfície.

Vórtice espiral: Energia verde roda em espiral convergindo para o centro, acelerando à medida que se aproxima. Cria o efeito de sucção dimensional característico de um portal ativo.

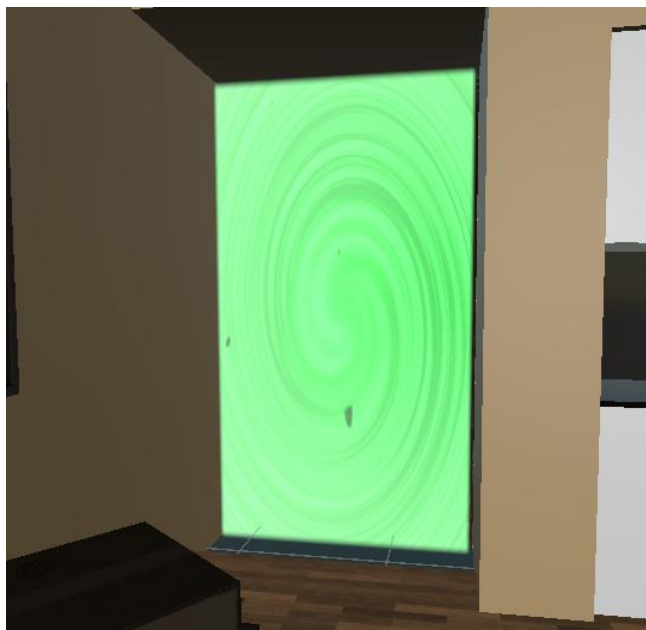
Linhas de vento: Riscos brancos e cinzentos que se movem do exterior para o interior do portal ao longo de linhas radiais, reforçando visualmente a força de atração.

Fragmentos em órbita: Pequenas pedras/fragmentos que orbitam e rodam no interior do portal, complementando o efeito de vórtice. São um particle system separado do shader, sem relevância técnica para a implementação HLSL.

Glow ambiental: A luz verde emitida pelo portal contamina o ambiente envolvente — o chão e as superfícies próximas ficam iluminadas a verde, integrando o efeito na cena.

12.2 Teoria — Técnicas de Implementação

O portal dimensional é implementado num fragment shader único aplicado a um Quad. As técnicas centrais são: Fresnel para a borda luminosa, FBM (Fractional Brownian Motion) para o vórtice interior, Value Noise para os fragmentos em órbita, e um gradiente radial para o núcleo central.



12.2.1 Borda — Fresnel

O brilho nas arestas resulta do efeito Fresnel: a intensidade é máxima quando a normal é perpendicular ao vetor de vista (bordas do objeto) e mínima quando está alinhada com ele (centro). O parâmetro `_FresnelPower` controla a curvatura da transição:

```
float NdotV = saturate(dot(normal, viewDir));  
float fresnel = pow(1.0 - NdotV, _FresnelPower);  
col += _FresnelColor.rgb * fresnel;
```

12.2.2. Interior — FBM e Rotação de UV

O vórtice usa FBM (Fractional Brownian Motion) — empilha quatro oitavas de Value Noise com amplitudes e frequências crescentes. O resultado é um padrão orgânico e irregular, muito mais próximo do original do que uma simples rotação de UV. Antes de amostrar o FBM, as UV são rodadas com uma matriz 2D animada cujo ângulo de twist depende do raio — fazendo o centro rodar mais do que as bordas:

```
float fbm(float2 p)
{
    float v = 0.0;
    v += 0.5000 * valueNoise(p); p = p * 2.02;
    v += 0.2500 * valueNoise(p); p = p * 2.03;
    v += 0.1250 * valueNoise(p); p = p * 2.01;
    v += 0.0625 * valueNoise(p);
    return v;
}
```

```
float twistAngle = radius * _SwirlTwist + t * _SwirlSpeed;
float s_twist = sin(twistAngle);
float c_twist = cos(twistAngle);
float2x2 rotTwist = float2x2(c_twist, -s_twist, s_twist, c_twist);
float2 swirlUV = mul(rotTwist, centered) * _SwirlScale;
swirlUV += float2(t * _InwardSpeed * 0.5, t * _InwardSpeed * 0.7);
float swirlNoise = fbm(swirlUV * _SwirlScale);
swirlNoise = smoothstep(0.2, 0.8, swirlNoise);
```

12.2.3 Fragmentos em Órbita — Value Noise

Os fragmentos são gerados no fragment shader com Value Noise de alta frequência. Uma segunda matriz de rotação mais lenta anima o movimento orbital. O produto de dois noisemaps e um smoothstep com limiar alto (0.75–0.8) garante que apenas uma fração mínima de pixels se torna fragmento:

```
float2 debrisUV = mul(rot, centered) * 15.0;
debrisUV += float2(t * 2.0, t * 1.5);
float debrisNoise = valueNoise(debrisUV);
debrisNoise *= valueNoise(debrisUV + 5.5);
float debris = smoothstep(0.75, 0.8, debrisNoise);

col = lerp(col, _DebrisColor.rgb, debris * 0.8);
```

12.2.4 Núcleo Central

O núcleo é um gradiente radial gerado com smoothstep sobre o raio, elevado a potência para concentrar o brilho no centro:

```
float coreGlow = 1.0 - smoothstep(0.0, _CoreSize, radius);
coreGlow = pow(coreGlow, 2.0);

col += _CoreColor.rgb * coreGlow;
```

12.2.5 Alpha Border

Para evitar bordas cortadas no Quad, o alpha é calculado com quatro smoothsteps sobre as UV — um por lado — que criam um fade suave de 2% nas extremidades:

```
float alphaBorder = smoothstep(0.0, 0.02, i.uv.x) * smoothstep(1.0, 0.98, i.uv.x) *  
                    smoothstep(0.0, 0.02, i.uv.y) * smoothstep(1.0, 0.98, i.uv.y);
```

12.3 Implementação

A implementação seguiu a teoria descrita acima. O shader Custom/Stencil/FrankStonePortal é um fragment shader único, transparente, aplicado a um Quad (Queue=Geometry+2, Blend SrcAlpha OneMinusSrcAlpha, ZWrite Off). O bloco Stencil com Ref 1 e Comp Always marca os pixels do portal no stencil buffer. Abaixo o código completo:



Figura — Referência: portal em The Casting of Frank Stone

<https://www.youtube.com/watch?v=3d0qQ4s6BCY>

13. Testes em Hardware Real – Meta Quest (Android)

Durante o desenvolvimento, o projeto foi testado em VR através do Unity Editor no PC com os Meta Quest ligados por cabo (Unity Link). Neste modo, os shaders correm na GPU do computador sob DirectX 11. Por pedido do professor, o projeto foi posteriormente compilado diretamente para Android e instalado nos óculos, passando a executar de forma autónoma na GPU integrada dos Meta Quest.

Esta mudança de plataforma introduziu diferenças visuais notáveis. A razão fundamental é que no PC os shaders compilam para DirectX 11 (HLSL nativo), enquanto nos óculos compilam para OpenGL ES, o standard gráfico do Android. O OpenGL ES é um subconjunto mais limitado — não suporta todas as funcionalidades do DirectX 11 — e a GPU Adreno dos Meta Quest é significativamente menos potente do que uma GPU desktop.

13.1 Kerr – GrabPass não suportado em OpenGL ES

O GrabPass usado para capturar o frame buffer e criar o efeito de lensing gravitacional não existe no OpenGL ES. Na build Android, o interior do buraco negro surge como uma esfera cinzenta plana — sem qualquer distorção do fundo. O disco de acreção continua a renderizar, mas o efeito de lente gravitacional desaparece completamente. Além disso, o raymarcher com 100 iterações por fragmento causou frame drops notáveis na GPU Adreno.



Figura — Kerr nos óculos: disco de acreção visível, mas sem lensing do fundo

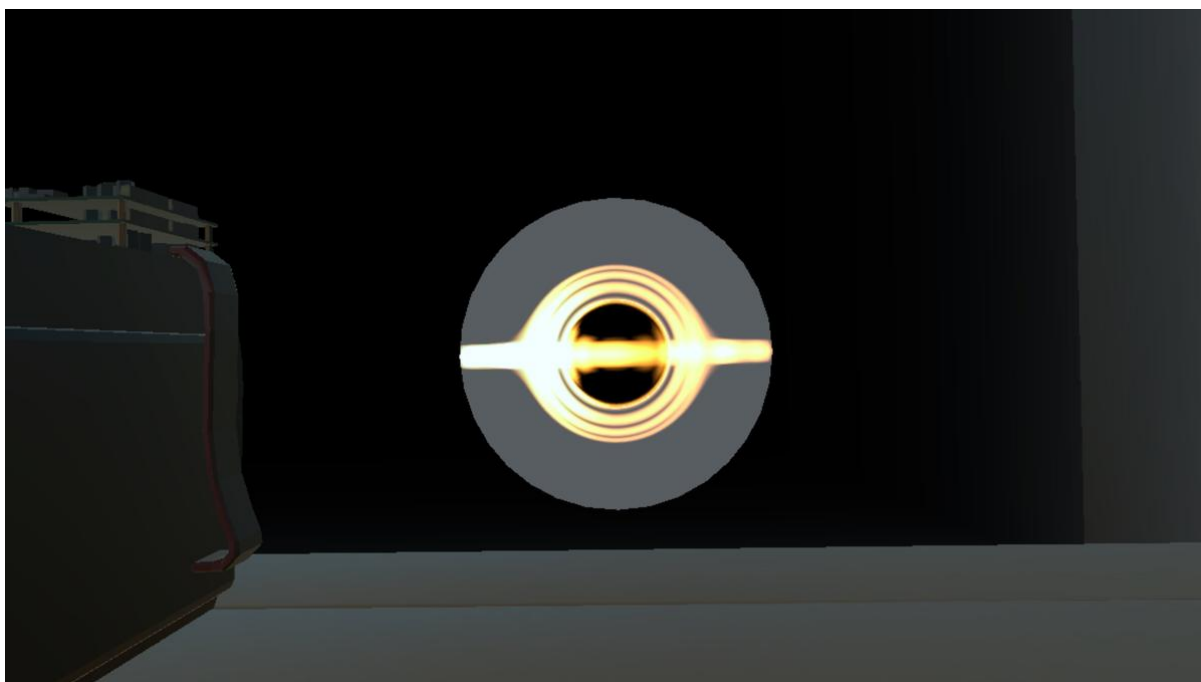


Figura - Kerr visto de outro ângulo; esfera cinzenta sem distorção de background

13.2 EnergyShield – Geometry Shader ignorado

A diretiva `#pragma target 4.0` com `FallBack Off` requer suporte a geometry shaders que não está disponível em OpenGL ES. Na build Android, o escudo renderizou como uma semiesfera sólida e flat — sem o ripple geométrico de impacto — já que o geometry shader foi silenciosamente ignorado pelo driver.

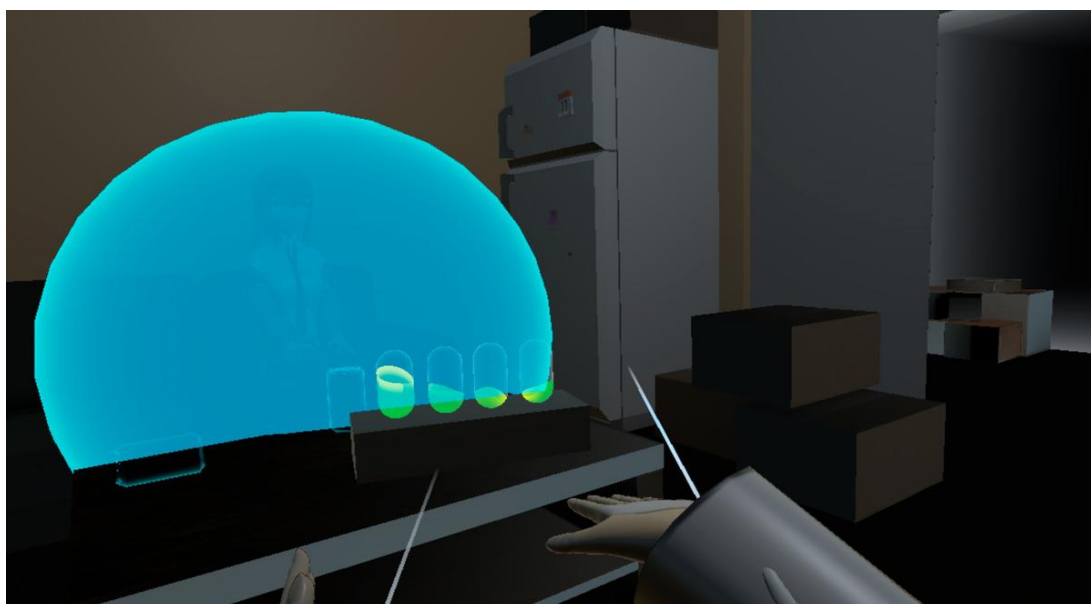


Figura - EnergyShield nos óculos: semiesfera sólida flat sem ondulação de impacto

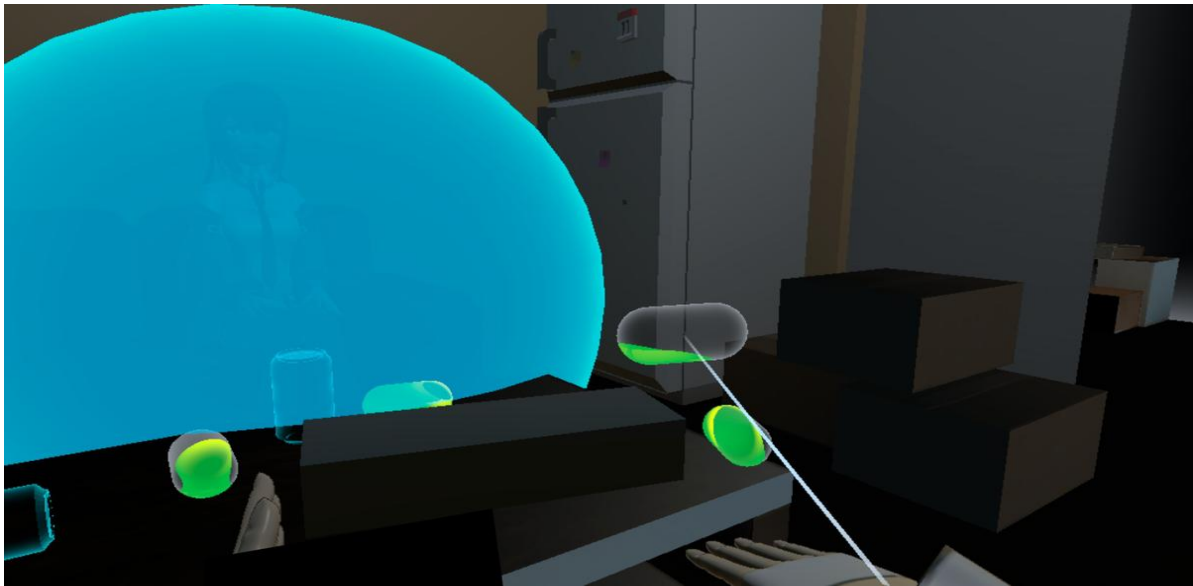


Figura - Vista próxima do escudo; ausência total do ripple geométrico

13.3 CCTV Glitch e ReadingSteiner – Post-process ausente

Os efeitos de pós-processamento dependem de OnRenderImage na câmara principal. Numa build Android para Meta Quest, a câmara é gerida pelo XR runtime dos óculos e não pela câmara Unity normal, pelo que OnRenderImage não é chamado e os efeitos simplesmente não aparecem.

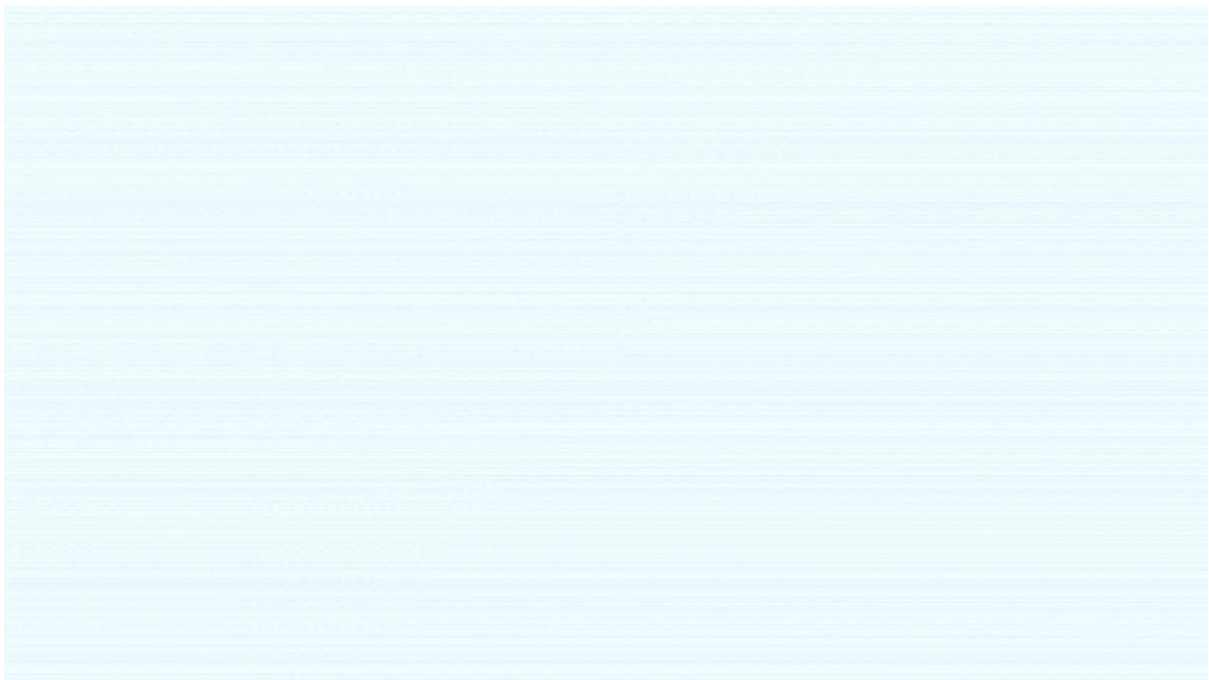


Figura - Ecrã sem qualquer efeito de pós-processamento ativo nos óculos

13.4 StencilPortal – Ordem de draw calls instável

O OpenGL ES não garante a mesma ordem de draw calls que o DirectX. As queues $Queue=Geometry+1$ e $Queue=Geometry+2$ que asseguram a sequência correta do stencil no PC podem não ser respeitadas em Android, tornando o objeto X-Ray visível incorretamente ou impossível de ver dentro do portal.

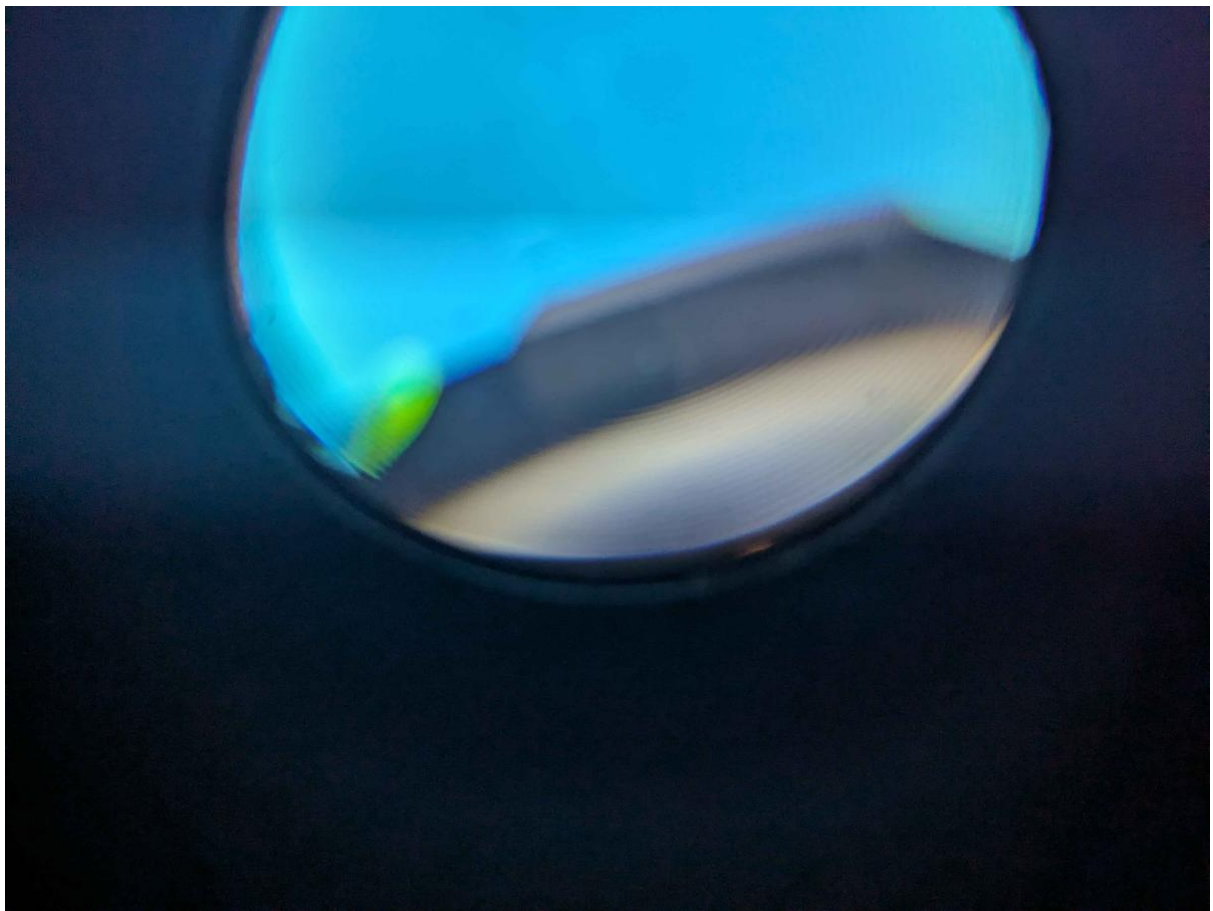


Figura - Portal nos óculos; comportamento do stencil instável em OpenGL ES

13.5 Cena Geral

A cena geral nos óculos manteve a maioria dos shaders funcionais — Hologram, RadioactiveLiquid e o ambiente base renderizaram corretamente. As diferenças concentraram-se nos shaders que dependem de funcionalidades específicas do DirectX 11 ou do pipeline de câmara do Unity Editor.



Figura - Vista geral da cena nos óculos; hologramas e liquid shader funcionais

14. Conclusão

Este trabalho cobre os seguintes tipos de shader: vertex (Hologram, EnergyShield), fragment (todos), geometry (EnergyShield), com hull documentado como descartado por compatibilidade. As técnicas implementadas incluem: pós-processamento (CCTV, ReadingSteiner), raymarching volumétrico (Kerr), Fresnel/rim (Hologram, EnergyShield, Radioactive, StencilWorld), stencil buffer (Radioactive, Portal), GrabPass com distorção, noise procedural (Value Noise com smoothstep cúbico), abertura cromática, geometry displacement dinâmico e as conversões GLSL→HLSL (random e smoothNoise).